Testing Foundations

SE 413/513 — Software Testing

Course Reading Material

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasaka@gvsu.edu.

Please cite [1] this material when using or reproducing it.

Unit: Testing Terminology, Principles, Verification and Validation, and the Testing Lifecycle

Contents

1 Learning Objectives

After completing this reading, you should be able to:

- Explain why systematic software testing matters, using real-world examples of what happens when it fails.
- Distinguish between an **error**, a **fault** (defect/bug), and a **failure**.
- Explain the difference between **verification** and **validation**.
- State and apply the seven fundamental principles of software testing.
- Classify defects by **severity** and **priority**, and describe a typical defect life cycle.
- Explain why the cost of fixing a defect grows the later it is discovered.
- Describe the levels of testing (unit, integration, system, acceptance) and how they relate to the V-model.
- Describe the phases of the Software Testing Life Cycle (STLC).
- Recognize common misconceptions about what testing can and cannot guarantee.

2 Why Do We Test Software?

Software testing is the process of evaluating a system to determine whether it meets specified requirements and to identify defects before users do. [4, 7] It is easy to treat testing as a bureaucratic afterthought — something to do once the "real" work of writing code is finished. Professional software engineering practice treats it very differently: testing is one of the primary ways an engineering team manages risk.

Consider what happens when testing fails. Some of the most expensive and dangerous incidents in software history trace back to defects that testing did not catch, or to teams that tested the wrong thing.

Real-World Case Study

Knight Capital Group (August 1, 2012). [9] A deployment engineer pushed new trading code to 7 of 8 production servers. The 8th server still had a dormant, seven-year-old test module that reused an old order-tag flag. When markets opened, that flag reactivated the obsolete code on the 8th server, which began sending roughly 4 million erroneous orders into the market. In just 45 minutes, Knight Capital lost approximately $440 million and nearly went bankrupt. The root cause was not a single "bug" in the traditional sense — it was a deployment and configuration-management failure that a disciplined testing and release process should have caught.

This example illustrates a theme that runs through this entire reading: **a defect is cheap to fix when it is caught early, and can be catastrophic when it is caught late** — or not caught at all. We will return to this idea more precisely in Section 7.

Key Idea

Testing is not something you do *to* a finished product. It is a discipline you apply *throughout* the software lifecycle to reduce the risk of releasing software that does not do what it is supposed to do.

3 Core Terminology: Error, Fault, and Failure

Casual conversation uses the word "bug" for almost anything that goes wrong. Software engineering distinguishes three related but different concepts. Getting this vocabulary right matters, because a test plan, a defect report, and a root-cause analysis all depend on knowing which of these three things you are actually talking about.

- **Error (or mistake):** A human action that produces an incorrect result. A developer misreads a specification, misunderstands a requirement, or simply makes a typo. The error happens in a person's head (or fingers), not in the software itself.
- **Fault (or defect, or "bug"):** A flaw in the software — in code, in a design document, or in documentation — that results from an error. A fault is a static, latent property of the artifact. It can sit in the codebase for a long time without causing any visible problem.
- **Failure:** An observable deviation of the system from its expected behavior, occurring when a fault is actually executed (or triggered) under specific conditions.

Key Idea

Error → Fault → Failure. A human **error** introduces a **fault** into the software. That fault causes a **failure** only when the program executes the faulty code *and* the conditions are right to expose it. A fault with no triggering conditions may never cause a failure at all — which is exactly why testing (deliberately trying to trigger faults) is necessary, rather than just waiting for users to report problems.

The Python example below shows this chain concretely. Suppose a developer misunderstands a billing requirement (the *error*): they believe that a late fee can never be negative, so they never considered what happens if `days_late` is negative (for example, due to a clock-synchronization issue upstream). This misunderstanding is encoded as a *fault* in the function:

```
def calculate_late_fee(days_late, daily_rate=1.50, max_fee=25.00):
    """Return the late fee for an overdue library book."""
    fee = days_late * daily_rate
    if fee > max_fee:
        return max_fee
    return fee
```

For every ordinary input the function behaves correctly:

```
>>> calculate_late_fee(3)
4.5
>>> calculate_late_fee(30)
25.0
```

The fault is invisible here — these test cases all pass. It only becomes a *failure* when an unusual input reaches the function:

```
>>> calculate_late_fee(-4)
-6.0
```

A negative fee makes no business sense, but nothing in the code prevents it. The fault (missing input validation) was present the whole time; it just needed the right triggering condition (a negative value of `days_late`) to produce an observable failure.

Tip

When you write a defect report, be precise about which of the three concepts you are describing. "The failure was a negative late fee shown to the customer; the fault was missing input validation in `calculate_late_fee()`; the underlying error was an incomplete requirements assumption about how `days_late` could arrive." This precision is what lets a team fix the actual fault instead of just patching one symptom.

4 Verification and Validation

Two questions sound similar but are fundamentally different, and confusing them is one of the most common sources of expensive mistakes in software engineering:

- **Verification:** "Are we building the product *right*?" Does the software conform to its specification? Does each component do what its design document says it should do?
- **Validation:** "Are we building the *right* product?" Does the software satisfy the actual needs of its users and stakeholders, in the real world it will operate in?

Key Idea

It is entirely possible to pass verification and still fail validation. A component can perfectly implement its specification, and the software can still be wrong, if the specification itself did not capture what was actually needed.

The following simplified Python example illustrates how this happens in practice: two components are each individually correct with respect to their own interface contract, yet the integrated system is wrong.

```python
# navigation_module.py
# Written by Team A. Expects thrust values in newton-seconds (metric),
# per the interface specification both teams agreed to.
def compute_trajectory_correction(thrust_newton_seconds):
    return thrust_newton_seconds * 0.0142  # simplified correction factor
```

```python
# ground_software.py
# Written by Team B. Computes thruster performance and passes it to
# navigation_module. Team B's internal tools use pound-force-seconds
# (imperial), and nobody converted the value before the call.
from navigation_module import compute_trajectory_correction
```

```python
def send_correction(thrust_pound_force_seconds):
    # BUG: passing imperial units into a function that expects metric
    return compute_trajectory_correction(thrust_pound_force_seconds)
```

Each function, in isolation, is easy to *verify*: given a documented input, does it produce the documented output? Both teams' unit tests for their own module can pass. But the system as a whole fails *validation*: it does not do what the mission actually needs, because the two components disagree about what a "thrust value" means.

Real-World Case Study

Mars Climate Orbiter (1999). [8] NASA's $327.6 million Mars Climate Orbiter mission was lost when it approached Mars at the wrong altitude and disintegrated in the atmosphere. The root cause: Lockheed Martin's ground software calculated thruster performance data in pound-force-seconds (imperial units), while the JPL navigation software that consumed that data expected newton-seconds (metric units), per the interface specification. Each side's software worked correctly against its own assumptions — the failure was an unverified interface between two components, not a coding typo. It remains one of the most cited examples of why interface contracts must be checked (verified), not just assumed.

Common Pitfall

"It passed all its tests" is not the same claim as "it is correct." A test suite can only verify a system against the specification (or against the tests themselves). If the specification is wrong, incomplete, or ambiguous, a fully verified system can still be validated as unfit for its real purpose.

5 The Seven Principles of Software Testing

These seven principles, widely taught in the software testing community (and codified by organizations such as ISTQB), summarize decades of hard-won experience about what testing can and cannot do. [5] Treat them as a mental checklist whenever you design a test strategy.

5.1 Principle 1: Testing Shows the Presence of Defects, Not Their Absence

Running tests and finding no failures does not prove the software is defect-free. It only shows that, for the inputs and conditions you tried, no failure was observed. There may be faults lurking in code paths your tests never exercised.

5.2 Principle 2: Exhaustive Testing Is Impossible

Except for the most trivial programs, it is not feasible to test every possible input, every possible state, and every possible sequence of operations. Consider a function that accepts two 32-bit integers: there are already over 1.8×10^{19} possible input pairs. Testing must therefore be a matter of choosing a representative, risk-informed subset of all possible scenarios, using techniques like equivalence partitioning and boundary-value analysis (covered in later weeks).

5.3 Principle 3: Early Testing Saves Time and Money

Defects are cheaper to find and fix the earlier they are discovered in the development lifecycle. A misunderstood requirement caught in a design review costs a conversation; the same misunderstanding caught after release can cost a full re-engineering effort, a patch release, and reputational damage. This is often called **shift-left testing**: moving testing activities as early as possible, rather than treating testing as a phase that only starts once coding is "done."

5.4 Principle 4: Defect Clustering

In practice, most defects in a system are found concentrated in a small number of modules. [5] This mirrors the Pareto principle ("80/20 rule"): a rough rule of thumb is that around 80% of defects are found in about 20% of modules. Modules that are complex, frequently modified, or written under time pressure are good candidates for extra testing attention.

5.5 Principle 5: The Pesticide Paradox

If you run the exact same set of tests repeatedly, they eventually stop finding new defects — just as insects eventually develop resistance to a pesticide used repeatedly. To keep finding new defects, test cases must be regularly reviewed, revised, and extended. [5]

5.6 Principle 6: Testing Is Context-Dependent

There is no single "correct" way to test all software. The testing strategy for a safety-critical medical device is (and must be) very different from the testing strategy for a marketing landing page. Risk tolerance, regulatory requirements, and the cost of failure all shape how much testing is appropriate and what kind.

Real-World Case Study

Therac-25 (1985–1987). [6] The Therac-25 was a radiation therapy machine whose software controlled beam intensity and mode. Unlike its predecessor, it removed hardware safety interlocks and relied on software alone to prevent the machine from firing a high-power electron beam directly at a patient without the beam spreader in place. A race condition let a fast-typing, experienced operator switch between treatment modes during an eight-second data-entry window in a way the software did not anticipate, bypassing the safety check. At least six patients received massive radiation overdoses, and at least three died. The Therac-25 is the canonical example of why safety-critical software testing must go far beyond "does it work under normal use," and why software-only safety mechanisms (with no hardware backup) demand a much higher standard of context-appropriate testing.

5.7 Principle 7: Absence-of-Errors Is a Fallacy

Even if you find and fix every defect you can locate, that alone does not guarantee success. If the system does not fulfill user needs, or does not meet the actual operational requirements, it can still fail — even with zero known bugs. This connects directly back to validation (Section 4): a system with no known defects can still be the wrong system.

Real-World Case Study

Ariane 5 Flight 501 (1996). [3] 37 seconds after launch, the European Space Agency's Ariane 5 rocket self-destructed, destroying its $370+ million payload. The cause was a 64-bit floating-point value (related to horizontal velocity) being converted into a 16-bit signed integer in the inertial reference system, causing an overflow. The module had been reused, unchanged, from the Ariane 4 rocket, where the same value could never exceed the 16-bit range given that rocket's flight profile. It was never re-validated against Ariane 5's different, faster trajectory — and the module kept running for a phase of flight where its output was not even needed, so a defect that could have been non-fatal instead triggered a shutdown of both (redundant) inertial reference units. The individual module had a clean track record on Ariane 4 ("zero known errors"), but that track record was irrelevant to whether it was validated for its new context.

Key Idea

Read together, the seven principles say something quite humbling: testing can build justified confidence, but it can never provide proof of correctness. Good testing is about managing risk intelligently, not about reaching some mythical state of "fully tested."

6 Defects: Classification and Life Cycle

Once a failure is observed, it needs to be reported, tracked, and resolved. Most teams use a defect-tracking system (Jira, GitHub Issues, Bugzilla, and similar tools) that expects each defect report to carry certain structured information.

6.1 Severity vs. Priority

These two attributes are frequently confused, but they answer different questions.

- **Severity** measures the *technical impact* of a defect on the system: how badly does it break functionality, data integrity, or safety?
- **Priority** measures the *urgency* of fixing it: how soon, relative to other work, should this defect be addressed?

A defect can be high severity but low priority (a crash in a rarely used administrative tool that has an easy workaround), or low severity but high priority (a misspelled company name on the homepage — functionally harmless, but embarrassing enough to fix immediately).

Level	Example (Severity)	Example (Priority)
Critical / Urgent	System crashes; data loss; security breach; safety hazard	Must be fixed before the next release, even if it delays shipping
High	Major feature is broken with no workaround	Should be fixed in the current sprint
Medium	Feature works but with a usability problem or minor incorrect output	Scheduled for an upcoming release
Low / Trivial	Cosmetic issue, typo, minor UI misalignment	Fixed when convenient, or deferred indefinitely

6.2 A Typical Defect Life Cycle

Most defect-tracking workflows follow some variation of the following states:

1. **New:** The defect has just been reported.
2. **Assigned:** A developer or team has been designated to investigate it.
3. **Open / In Progress:** Someone is actively working on a fix.
4. **Fixed:** A code change addressing the defect has been made.
5. **Retest:** A tester (often someone other than the fixer) verifies the fix against the original failure.
6. **Closed:** The retest confirms the defect is resolved.
7. **Reopened:** The retest shows the defect is still present (or the fix introduced a new problem), sending it back to *Open*.

A defect report itself typically records: a unique ID, a short summary, the environment/version it was found in, precise steps to reproduce, the expected result, the actual result, severity, priority, and status. Precision in "steps to reproduce" is what separates a defect report a developer can act on from one that just says "it's broken."

Tip

When you file a defect, imagine you will never get to talk to the reporter again. Could someone else reproduce the failure using only what you wrote down? If not, add more detail — exact inputs, exact environment, exact sequence of actions.

7 The Cost of Defects

One of the most consistently reproduced findings in software engineering research is that **the cost of fixing a defect increases the later it is discovered in the development lifecycle**. A defect caught during requirements review might cost the time it takes to correct a document. The same conceptual mistake, if it survives all the way to a production incident, can cost emergency patches, customer-facing outages, lost revenue, and reputational damage that no patch can fully undo.

This is the direct link between Section 7 and the Knight Capital and Mars Climate Orbiter examples above: neither company lost money because a single line of code was "wrong" in some abstract sense. They lost money because a defect that could have been caught during design review, code review, or staged deployment testing instead reached production, where the cost of the same underlying mistake was orders of magnitude higher.

Key Idea

This is the economic argument for **shift-left testing** (Principle 3, Section 4.3): every phase you push a defect's discovery earlier into — requirements, design, code review, unit testing, integration testing, staging, production — tends to reduce the cost of fixing it, often dramatically.

8 Levels of Testing

Testing activities are usually organized into levels, each targeting a different scope of the system. You will study each of these levels, and the specific techniques used within them, in much greater depth later in the course — this section only introduces the vocabulary.

- **Unit testing:** Tests a single unit of code (typically a function, method, or class) in isolation, often replacing its dependencies with test doubles (as in the `MagicMock` reading).
- **Integration testing:** Tests how multiple units or components work together, focusing especially on the interfaces between them — precisely the kind of boundary where the Mars Climate Orbiter defect lived.
- **System testing:** Tests the complete, integrated system against its overall functional and non-functional requirements, typically in an environment resembling production.
- **Acceptance testing:** Validates the system against user or business needs, often performed by, or on behalf of, the customer, to decide whether the system is ready for release.

A minimal unit test for the `calculate_late_fee()` function from Section 3, using Python's built-in `unittest` framework, might look like this:

```python
import unittest

class TestLateFee(unittest.TestCase):
    def test_typical_fee(self):
        self.assertEqual(calculate_late_fee(3), 4.5)

    def test_fee_is_capped(self):
        self.assertEqual(calculate_late_fee(30), 25.00)

    def test_negative_days_should_not_be_negative_fee(self):
        # This test documents the requirement and currently FAILS,
        # exposing the fault discussed in Section 3.
        self.assertGreaterEqual(calculate_late_fee(-4), 0)

if __name__ == "__main__":
    unittest.main()
```

Notice the third test: it is written to describe what *should* happen, and it currently fails against the buggy implementation. A failing test that accurately captures a requirement is not a problem to hide — it is valuable information, and exactly the kind of signal Principle 1 (Section 4.1) describes: it shows the presence of a defect.

9 The V-Model and the Software Testing Life Cycle

The **V-model** is a classic way of visualizing how testing activities correspond to development activities, emphasizing that testing should be planned alongside each development phase, not bolted on afterward.

Development Activity (left side)	**Corresponding Test Activity (right side)**
Requirements analysis	Acceptance testing
System design	System testing
Architecture / high-level design	Integration testing
Module / detailed design	Unit testing

Each development activity on the left is paired with a testing activity on the right that verifies and validates it. Critically, the V-model implies that acceptance test cases can be drafted as soon as requirements are written — long before any code exists — because acceptance tests are validating the requirements, not the implementation.

The **Software Testing Life Cycle (STLC)** describes the phases testing itself goes through, in parallel with (not strictly after) development:

1. **Requirement analysis:** Testers review requirements to identify what is testable, and to catch ambiguity or missing information early (Principle 3).
2. **Test planning:** Deciding scope, approach, resources, schedule, and entry/exit criteria for testing.
3. **Test case design:** Writing detailed test cases and preparing test data, based on the requirements and design.
4. **Test environment setup:** Preparing the hardware/software environment the tests will run in.
5. **Test execution:** Running the tests, logging results, and filing defect reports for failures.
6. **Test cycle closure:** Evaluating results against exit criteria, documenting lessons learned, and archiving test artifacts for future reference.

10 Common Misconceptions

Common Pitfall

"100% code coverage means the code is bug-free." Code coverage measures which lines or branches were *executed* during testing, not whether the correct *outcomes* were actually checked. A test can execute every line of `calculate_late_fee()` while never asserting anything about negative input, and coverage tools will happily report 100%.

Common Pitfall

"Testing is QA's job, not mine." Modern software engineering treats quality as a shared responsibility across the whole team. Developers write unit tests and participate in code review; QA engineers design broader test strategies and may specialize in techniques developers do not; product stakeholders participate in acceptance testing and validation. No single role can single-handedly guarantee quality.

Common Pitfall

"If it passed all our tests, it's correct." As discussed throughout this reading (Principles 1 and 7, and the Ariane 5 and Mars Climate Orbiter case studies), passing tests demonstrates the *absence of the specific failures you checked for* — not the presence of correctness in any absolute sense.

11 A Practical Mental Model

When you see or hear:

> "This module passed code review and all its unit tests."

think:

> "It has been *verified* against its own specification. Has anyone *validated* that the specification itself is what the system actually needs?"

When you see:

> "We didn't find any bugs in this release."

think:

> "That means our tests found no failures — not that no faults exist. What parts of the input space or system state did we not exercise?"

When you see:

> "This defect is only in an edge case; it's probably not a big deal."

think:

> "Severity and priority are different questions. How bad would it be *if* it happened (severity), and separately, how urgently do we need to fix it (priority)?"

When you see:

> "We'll test it thoroughly before the next release."

think:

> "The cost of every defect we find today is lower than the cost of finding it tomorrow. What can shift left?"

12 Summary

The most important vocabulary from this reading is summarized below.

Term	Meaning
Error (mistake)	A human action that produces an incorrect result
Fault (defect / bug)	A flaw in software or documentation, resulting from an error
Failure	An observable deviation from expected behavior, caused by a fault being triggered
Verification	Checking that the product was built *right* (conforms to spec)
Validation	Checking that the *right* product was built (meets real needs)
Severity	The technical impact of a defect
Priority	The urgency of fixing a defect
Shift-left testing	Moving testing activities earlier in the lifecycle to reduce cost
Unit / Integration / System / Acceptance testing	The four classic levels of testing, increasing in scope
STLC	The phases the testing process itself moves through

The central idea is simple:

Key Idea

Testing builds justified confidence; it does not build proof. Every technique in this course — from unit tests with `MagicMock` to the testing principles in this reading — exists to help you find defects earlier, understand what your tests actually demonstrate, and reason clearly about the difference between "it works the way we specified" and "it works the way it needs to."

13 Practice Exercises

Exercise 1 — Error, Fault, Failure

Recall the `calculate_late_fee()` example from Section 3. Identify a second, different fault that could be introduced into that function by a plausible requirements misunderstanding (an *error*). Describe the fault in code terms, and describe a specific input that would trigger a *failure*.

Exercise 2 — Verification vs. Validation

A team builds a search feature that returns results in under 200ms, exactly as specified in the design document, and every unit test passes. After launch, users complain that the search results are frequently irrelevant. Was this a verification failure, a validation failure, or both? Justify your answer.

Exercise 3 — Severity and Priority

For each defect below, assign a severity (Critical/High/Medium/Low) and a priority (Critical/High-/Medium/Low), and briefly justify each rating:

1. A checkout page crashes for all users on the busiest shopping day of the year.
2. An internal admin report, used once a quarter, displays a date in the wrong format.
3. A rarely used keyboard shortcut in a code editor does nothing when pressed.

Exercise 4 — The Seven Principles

The Ariane 5 case study is most directly connected to which of the seven principles? Explain the connection in your own words, and identify a second principle that also applies to that case study.

Exercise 5 — Design Question

Consider the claim: "If our test suite has 100% code coverage, we don't need manual or exploratory testing." Do you agree or disagree? Use at least one concept from this reading (coverage vs. correctness, verification vs. validation, or one of the seven principles) to support your answer.

14 Discussion Questions

1. In your own words, what is the difference between a fault and a failure? Why does a fault sometimes go unnoticed for years?
2. Why can a system pass every verification check and still be the wrong system?
3. Which of the seven testing principles do you find most counterintuitive, and why?
4. How would you explain the difference between severity and priority to a non-technical stakeholder?
5. Why does the cost of a defect tend to rise the later it is discovered? What kinds of costs are involved besides engineering time?
6. In the Therac-25 case, the software had no hardware safety interlock to fall back on. What does that suggest about testing requirements for safety-critical systems specifically?
7. How does "testing is context-dependent" (Principle 6) apply differently to, say, a hobby project versus a piece of medical device software?
8. What information should always be included in a defect report, and why does precision matter so much?

9. What is "shift-left testing," and what are some concrete ways a team could shift testing earlier in a real project?
10. Can you think of a personal or well-known example (beyond the ones in this reading) where a system was verified correctly but failed validation, or vice versa?

15 Key Takeaways for SE 413/513

For software engineering practice, remember the following principles:

1. **Vocabulary is not pedantic — it is precision.** Distinguishing error, fault, and failure lets a team communicate exactly what happened and fix the right thing.
2. **Verification and validation ask different questions.** Passing every test does not guarantee you built the right product.
3. **Testing manages risk; it does not produce proof.** The seven principles exist to keep expectations about testing realistic and effective.
4. **Severity and priority are independent dimensions.** A defect's technical impact and its fix urgency should be assessed separately.
5. **Cost grows with delay.** The earlier a defect is found, the cheaper it is to fix — this is the core argument for testing early and often. [2]
6. **Testing is contextual.** The right amount and kind of testing depends on the risk, cost of failure, and domain of the system.
7. **Real incidents are case studies, not trivia.** Knight Capital, Mars Climate Orbiter, Therac-25, and Ariane 5 are not just interesting stories — each one maps directly onto a concept you will use for the rest of this course.

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] Boehm, B. W. (1981). *Software Engineering Economics*. Prentice-Hall.

[3] European Space Agency. (1996). *Ariane 5 Flight 501 Failure: Report by the Inquiry Board*.

[4] IEEE. (2022). *IEEE Std 29119-1-2022: IEEE Standard for Software and Systems Engineering—Software Testing—Part 1: General Concepts*. IEEE.

[5] International Software Testing Qualifications Board (ISTQB). (2018). *Certified Tester Foundation Level Syllabus, Version 2018*.

[6] Leveson, N. G., & Turner, C. S. (1993). An investigation of the Therac-25 accidents. *Computer*, 26(7), 18–41. https://doi.org/10.1109/MC.1993.274940

[7] Myers, G. J., Sandler, C., & Badgett, T. (2011). *The Art of Software Testing* (3rd ed.). Wiley.

[8] Mars Climate Orbiter Mishap Investigation Board. (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*. NASA.

[9] U.S. Securities and Exchange Commission. (2013). *SEC Charges Knight Capital With Violations of Market Access Rule*. SEC.